Solution: Word Ladder

Problem Statement

A `transformation sequence` is a sequence that starts from the word `beginWord`, ends with `endWord`, and contains a word from the given dictionary `wordList` in the middle i.e. a sequence of words `beginWord -> s1 -> s2 -> ... -> sk` such that:

- Each adjacent pair of words differs by a single letter.
- Each si for `1 <= i <= k` is in wordList. sk == endWord
- The `beginWord` does not need to be in wordList.

Given two strings, `beginWord` and `endWord`, and a `wordList`, return the number of words in the shortest transformation sequence from beginWord to endWord, or -1 if no such sequence exists.

Examples

- **Example 1**:
  - **Input**: startWord = "cat", endWord = "dog", wordList = ["cot", "dot", "dog"]
  - **Expected Output**: 4
  - **Justification**: The shortest transformation sequenc is "cat" -> "cot" -> "dot" -> "dog", which has 4 words.
- **Example 2**:
  - **Input**: startWord = "bit", endWord = "pog", wordList = ["but", "put", "pig", "pog", "big"]
  - **Expected Output**: 4
  - **Justification**: The shortest transformation sequenc is "bit" -> "big" -> "pig" -> "pog", which has 4 words.
- **Example 3**:
  - **Input**: startWord = "sing", endWord = "ring", wordList = ["ring"]
  - **Expected Output**: 2
  - **Justification**: The shortest transformation sequenc is "sing" -> "ring", which has 2 words.

Solution

To solve this problem, we will utilize the Breadth-First Search (BFS) algorithm. This approach is suitable because it efficiently explores all possible transformations from the start word to the target word, level by level, ensuring that the shortest path, if it exists, is found. The BFS algorithm is ideal for this scenario because it systematically explores adjacent nodes (words that differ by a single letter) before moving on to the next level of nodes. This method guarantees that the first time the target word is reached, it is done so using the minimum number of transformations. Our algorithm will use a queue to keep track of the words to be explored and a set to mark words that have been visited to avoid repetition and cycles.

The algorithm starts by adding the start word to the queue and marking it as visited. Then, for each word currently in the queue, it generates all possible one-letter transformations and checks if they are in the word list. If a transformation is found in the word list and has not been visited, it is added to the queue and marked as visited. This process continues until the target word is found or the queue is empty, indicating that no transformation sequence exists. By tracking the depth of the search, we can determine the length of the shortest transformation sequence.

Step-by-step Algorithm

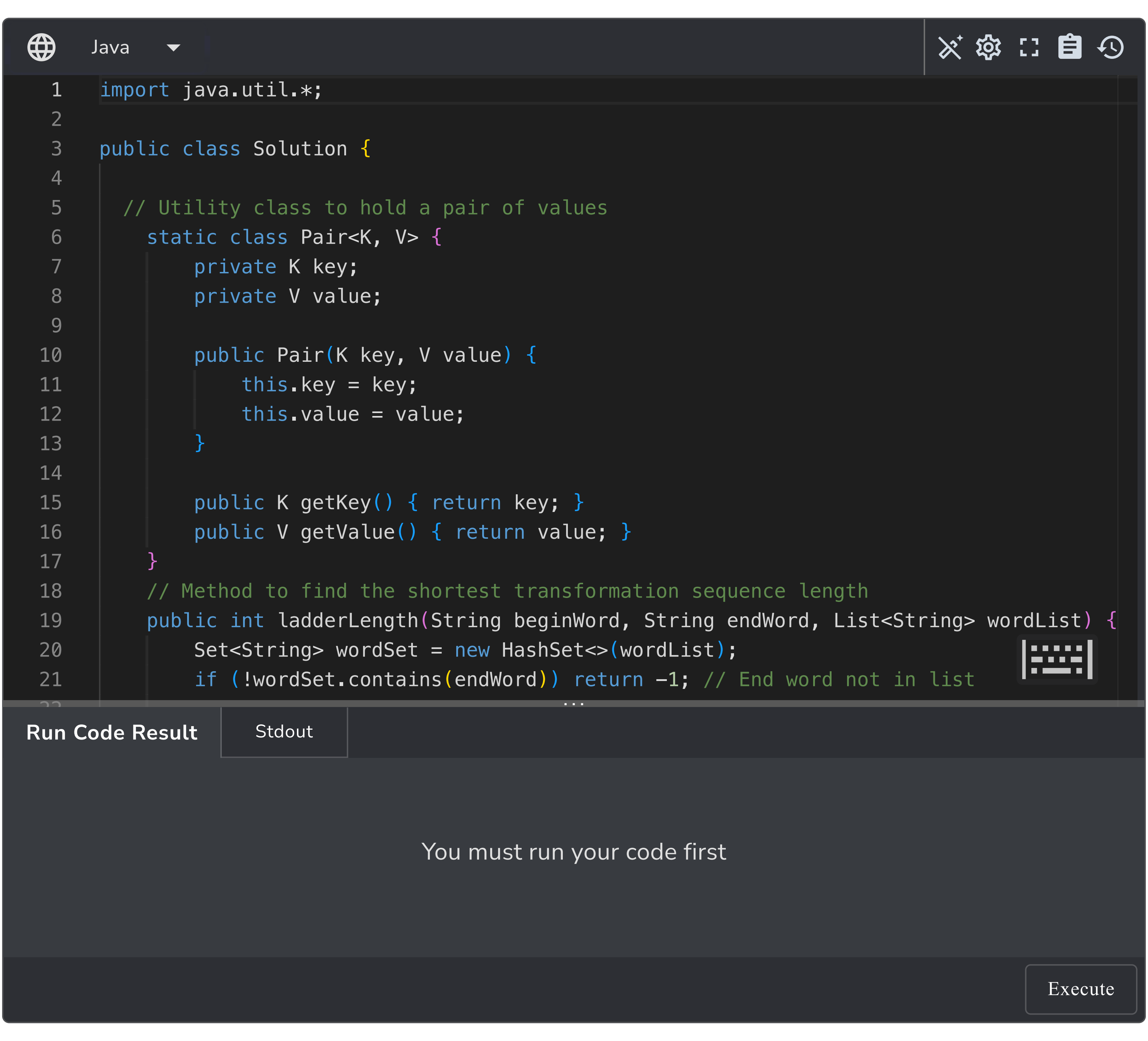
1. **Initialize Data Structures**:
   - Create a `HashSet` (or equivalent) to store all the words in the `wordList` for efficient access and to quickly check if a word exists in the list.
   - Initialize a `Queue` to perform the Breadth-First Search (BFS). Each element in the queue should be a pair (or equivalent) consisting of a word and its transformation distance from the `startWord`.
2. **Check for End Word**:
   - Check if the `endWord` exists in the `wordList`. If it does not, return -1 immediately since transformation is impossible without the `endWord` being in the list.
3. **Start BFS**:
   - Add the `startWord` to the queue with a transformation distance of 1 (since the `startWord` is the first step in the transformation sequence).
4. **Process the Queue**:
   - While the queue is not empty:
     - Dequeue the front pair to get the current word and its transformation distance.
     - If the current word is the `endWord`, return its transformation distance as the result.
     - Generate all possible transformations of the current word by changing each letter of the word to every other letter in the alphabet (from 'a' to 'z').
     - For each generated transformation:
       - If the transformation is present in the `wordList` (i.e., it's a valid word), add it to the queue with a transformation distance incremented by 1 and remove this word from the `wordList` or mark it as visited to prevent revisiting.
5. **Return Result**:
   - If the queue becomes empty and the `endWord` has not been reached, return -1 indicating that the transformation is not possible.

Algorithm Walkthrough

Let's consider the input: startWord = "bit", endWord = "pog", wordList = ["but", "put", "pig", "pog", "big"]

1. **Initialize Data Structures**:
   - Create a `HashSet` containing {"but", "put", "pig", "pog", "big"}.
   - Initialize a queue and add ("bit", 1) to it.
2. **Check for End Word**:
   - "pog" is in the `wordList`, so continue.
3. **Start BFS**:
   - The queue initially contains ("bit", 1).
4. **Process the Queue**:
   - Dequeue ("bit", 1):
     - Generate transformations of "bit": ("ait", "bit", "cit", ..., "zit"), ("bat", "bbt", ..., "bzt"), ("bia", "bib", ..., "biz").
     - Among these, "but" and "big" are in the `wordList`.
       - Enqueue ("but", 2) and remove "but" from the `wordList`.
       - Enqueue ("big", 2) and remove "big" from the `wordList`.
   - Dequeue ("but", 2):
     - Generate transformations of "but". Since "put" is a valid transformation and is in the `wordList`, enqueue ("put", 3) and remove "put" from the set.
   - Dequeue ("big", 2):
     - Generate transformations of "big". Since "pig" is a valid transformation and is in the `wordList`, enqueue ("pig", 3) and remove "pig" from the set.
   - Dequeue ("put", 3):
     - Generate transformations of "put". No valid transformation of "put" found in the "wordList".
   - Dequeue ("pig", 3):
     - Generate transformations of "pig". "pog" is a valid transformation and in the `wordList`, enqueue ("pog", 4) and remove "pog" from the set.
5. **Return Result**:
   - Dequeue ("pog", 4), it matches the `endWord`, and return 4 as the minimum number of steps required to transform "bit" to "pog".

Code



```java
import java.util.*;

public class Solution {

  // Utility class to hold a pair of values
    static class Pair<K, V> {
        private K key;
        private V value;

        public Pair(K key, V value) {
            this.key = key;
            this.value = value;
        }

        public K getKey() { return key; }
        public V getValue() { return value; }
    }
    // Method to find the shortest transformation sequence length
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> wordSet = new HashSet<>(wordList);
        if (!wordSet.contains(endWord)) return -1; // End word not in list
```

You must run your code first

Complexity Analysis

Time Complexity

- **Breadth-First Search (BFS)**: The algorithm performs a BFS, starting from the `beginWord` and exploring all possible one-letter transformations.
- **Worst-case Scenario**: In the worst case, we might have to explore all words in the `wordList`. For each word, we generate all possible one-letter transformations and check if they are in the `wordList`.
- **Generating Transformations**: For a word of length L and an alphabet size of (A) (26 for lowercase English letters), generating all possible one-letter transformations takes $O(L * A)$ time.
- **Checking Transformations**: With a hash set, checking if the transformation exists in the `wordList` takes $O(1)$ on average.
- **Overall Time Complexity**: Considering (N) as the number of words in the `wordList`, the overall worst-case time complexity is $O(N * L * A)$.

Space Complexity

- **Queue Storage**: The BFS queue can, in the worst case, hold all words in the `wordList`, leading to a space complexity of $O(N)$.
- **Visited Words**: Storing the visited words (or removing them from the `wordList` set) also requires $O(N)$ space.
- **Overall Space Complexity**: Thus, the overall space complexity is $O(N)$.

← Previous
Word Ladder (Hard)
Next →
Word Ladder II (Hard)
☐ Mark as Completed